

Image Processing and Analysis

Lecture 4、Image Enhancement in Frequency Domain

Fang Wan
School of Computer Science and Technology, UCAS
October 29, 2025

Outline

- 1 2-D Discrete Fourier Transform
- 2 Filtering in the Frequency Domain
- 3 Generating Filters Directly in the Frequency Domain
- 4 Sharpening Frequency Domain Filters
- 5 Obtaining Frequency Domain Filters from Spatial Filters

2-D Fourier Transform

- Any function that **periodically** repeats itself can be expressed as the **sum** of sines and/or cosines of different frequencies, each multiplied by a different coefficient (**Fourier series**).
- Even functions that are **not periodic** (but whose area under the curve is finite) can be expressed as the **integral** of sines and/or cosines multiplied by a weighting function (**Fourier transform**).
- The **frequency domain** refers to the plane of the two dimensional discrete Fourier transform of an image.
- The purpose of the Fourier transform is to represent a signal as a **linear combination of sinusoidal signals of various frequencies**.

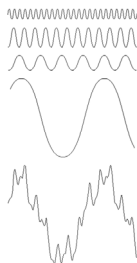


FIGURE 4.1 The function at the bottom is the sum of the four functions above it. Fourier's idea in 1807 that periodic functions could be represented as a weighted sum of sines and cosines was met with skepticism.

2-D Continuous Fourier Transform

- The **one-dimensional** Fourier transform and its inverse

- Fourier transform

$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi ux} dx, \text{ where } j = \sqrt{-1}$$

- Inverse Fourier transform:

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{j2\pi ux} du \quad e^{j\theta} = \cos \theta + j \sin \theta$$

- The **two-dimensional** Fourier transform and its inverse

- Fourier transform

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-j2\pi(ux+vy)} dx dy$$

- Inverse Fourier transform:

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{j2\pi(ux+vy)} du dv$$

2-D Discrete Fourier Transform

- The **one-dimensional** Discrete Fourier transform (DFT) and its inverse

- Fourier transform

$$F(u) = \frac{1}{M} \sum_{x=0}^{M-1} f(x) e^{-\frac{j2\pi ux}{M}} \quad \text{for } u = 0, 1, 2, \dots, M-1$$

- Inverse Fourier transform:

$$f(x) = \sum_{u=0}^{M-1} F(u) e^{\frac{j2\pi ux}{M}} \quad \text{for } x = 0, 1, 2, \dots, M-1$$

- Since $e^{j\theta} = \cos \theta + j \sin \theta$, then DFT can be redefined as

$$F(u) = \frac{1}{M} \sum_{x=0}^{M-1} f(x) \left[\cos \frac{2\pi ux}{M} - j \sin \frac{2\pi ux}{M} \right] \\ \text{for } u = 0, 1, 2, \dots, M-1$$

- Frequency (time) domain:** the domain (values of u) over which the values of $F(u)$ range; because u determines the frequency of the components of the transform.
- Frequency (time) component:** each of the M terms of $F(u)$.

2-D Discrete Fourier Transform

- $F(u)$ can be expressed in polar coordinates:

$$F(u) = |F(u)|e^{j\phi(u)}$$

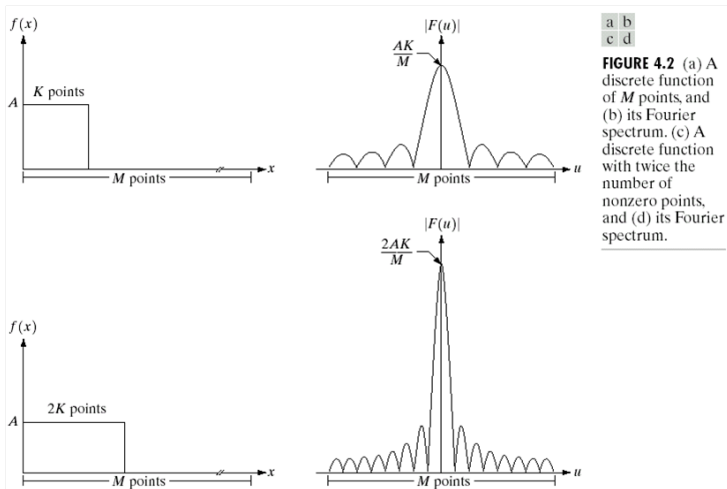
where $|F(u)| = [R(u)^2 + I(u)^2]^{\frac{1}{2}}$ (magnitude or spectrum)

$$\phi(u) = \tan^{-1}\left[\frac{I(u)}{R(u)}\right] \quad (\text{phase angle or phase spectrum})$$

- $I(u)$: the imaginary part of $F(u)$.
- $R(u)$: the real part of $F(u)$.
- Power spectrum

$$P(u) = |F(u)|^2 = R^2(u) + I^2(u)$$

2-D Discrete Fourier Transform



2-D Discrete Fourier Transform

- The **two-dimensional** Fourier transform and its inverse

- Fourier transform (**discrete case**) DTC

$$F(u, v) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

for $u = 0, 1, 2, \dots, M-1, v = 0, 1, 2, \dots, N-1$

- Inverse Fourier transform:

$$f(x, y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

for $x = 0, 1, 2, \dots, M-1, y = 0, 1, 2, \dots, N-1$

- u, v : the transform or frequency variables
- x, y : the spatial or image variables

2-D Discrete Fourier Transform

- We define the Fourier spectrum, phase angle, and power spectrum as follows:

$$|F(u, v)| = [R^2(u, v) + I^2(u, v)]^{\frac{1}{2}} \quad (\text{*spectrum*})$$

$$\phi(u, v) = \tan^{-1}\left[\frac{I(u, v)}{R(u, v)}\right] \quad (\text{*phase angle*})$$

$$P(u, v) = |F(u, v)|^2 = R^2(u, v) + I^2(u, v) \quad (\text{*power spectrum*})$$

- $I(u, v)$: the imaginary part of $F(u, v)$.
- $R(u, v)$: the real part of $F(u, v)$.

Properties of 2-D DFT

- Time-shifting

$$\mathfrak{F}[f(x - x_0, y - y_0)] = F(u, v)e^{-j2\pi(\frac{ux_0}{M} + \frac{vy_0}{N})}$$

- Frequency shifting

$$\mathfrak{F}[f(x, y)e^{-j2\pi(\frac{u_0x}{M} + \frac{v_0y}{N})}] = F(u - u_0, v - v_0)$$

$$\mathfrak{F}[f(x, y)(-1)^{x+y}] = F(u - \frac{M}{2}, v - \frac{N}{2})$$

- Average and Symmetry

$$F(0,0) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \quad (\text{average})$$

$$F(u, v) = F^*(-u, -v) \quad (\text{conjugate symmetric})$$

$$|F(u, v)| = |F(-u, -v)| \quad (\text{symmetric})$$

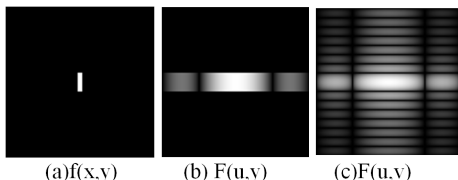
Properties of 2-D DFT (cont.)

- Separability

$$\begin{aligned}
 F(u, v) &= \mathfrak{F}[f(x, y)] \\
 &= \frac{1}{N} \sum_y \left[\frac{1}{M} \sum_x f(x, y) \exp \left(-j2\pi \frac{xu}{M} \right) \right] \exp \left(-j2\pi \frac{yv}{N} \right) \\
 &= \frac{1}{N} \sum_y F(u, y) \exp \left(-j2\pi \frac{yv}{N} \right)
 \end{aligned}$$

The 2D DFT $F(u, v)$ can be obtained by

- 1 Taking the 1D DFT of every row of image $f(x, y)$, $F(u, y)$
- 2 The 1D DFT of every column of $F(u, y)$

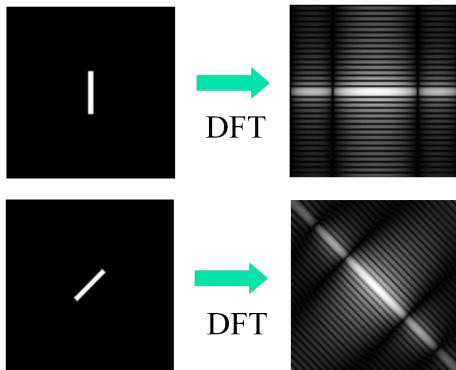


Properties of 2-D DFT (cont.)

- Rotation

Let $x = r \cos \theta, y = r \sin \theta, u = \omega \cos \varphi, v = \omega \sin \varphi$

$$f(r, \theta + \theta_0) \Leftrightarrow F(\omega, \varphi + \theta_0)$$



Properties of 2-D DFT (cont.)

- Periodicity

$$f(x, y) = f(x + M, y) = f(x, y + N) = f(x + M, y + N)$$

$$F(u, v) = F(u + M, v) = F(u, v + N) = F(u + M, v + N)$$

- Linearity

$$\mathfrak{F}(af(x, y) + bg(x, y)) = a\mathfrak{F}(f(x, y)) + b\mathfrak{F}(g(x, y))$$

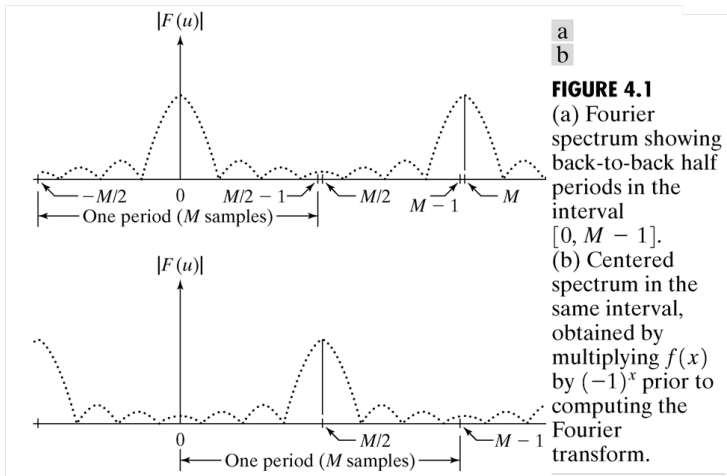
- Differentiation

$$\mathfrak{F}\left(\frac{\partial^n f(x, y)}{\partial x^n}\right) = (j2\pi u)^n \mathfrak{F}(f(x, y)) = (j2\pi u)^n F(u, v)$$

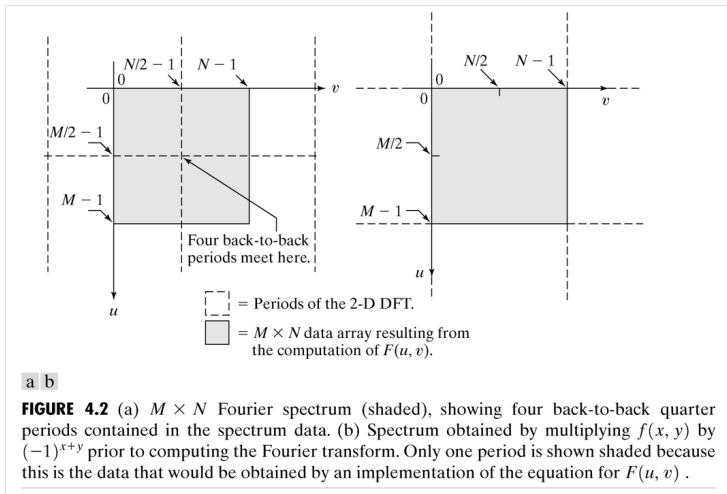
$$\mathfrak{F}((-j2\pi u)^n f(x, y)) = \frac{\partial^n F(u, v)}{\partial u^n}$$

$$\mathfrak{F}(\nabla^2 f(x, y)) = -4\pi^2(u^2 + v^2)F(u, v)$$

2-D Discrete Fourier Transform



2-D Discrete Fourier Transform (cont.)



Properties of 2-D DFT (cont.)

- Convolution

$$\mathfrak{F}(f(x, y) * g(x, y)) = F(u, v)G(u, v)$$

$$\mathfrak{F}(f(x, y)g(x, y)) = F(u, v) * G(u, v)$$

- Correlation

$$\mathfrak{F}(f(x, y) \circ g(x, y)) = F^*(u, v)G(u, v)$$

$$\mathfrak{F}(f(x, y) \circ f(x, y)) = |F(u, v)|^2$$

$$\mathfrak{F}(f^*(x, y)g(x, y)) = F(u, v) \circ G(u, v)$$

$$\mathfrak{F}(|f(x, y)|^2) = F(u, v) \circ F(u, v)$$

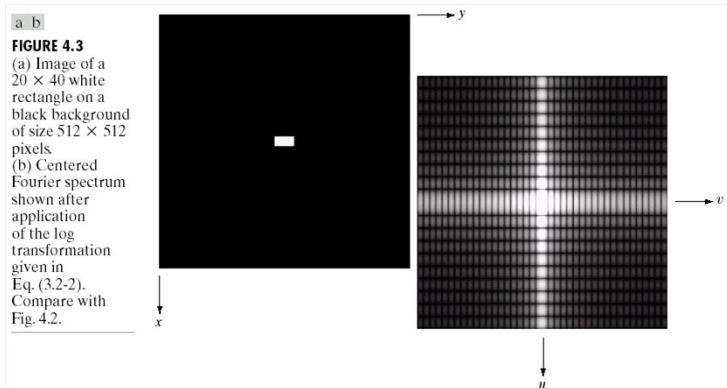
- Similarity

$$\mathfrak{F}(f(ax, by)) = \frac{1}{|ab|} F\left(\frac{u}{a}, \frac{v}{b}\right)$$

Some useful FT pairs

- $\delta(x, y) \Leftrightarrow 1$
- $A2\pi\sigma^2\exp(-2\pi^2\sigma^2(x^2 + y^2)) \Leftrightarrow A\exp(-\frac{(u^2 + v^2)}{2\sigma^2})$
 $\exp(-\pi(x^2 + y^2)) \Leftrightarrow \exp(-\pi(u^2 + v^2))$
- $\cos(2\pi u_0x + 2\pi v_0y) \Leftrightarrow \frac{1}{2}[\delta(u + u_0, v + v_0) + \delta(u - u_0, v - v_0)]$
- $\sin(2\pi u_0x + 2\pi v_0y) \Leftrightarrow \frac{1}{2}j [\delta(u + u_0, v + v_0) - \delta(u - u_0, v - v_0)]$

2-D Discrete Fourier Transform



2-D DFT in Matlab

- The DFT and its inverse are obtained in practice using a Fast Fourier Transform(FFT) algorithm. The FFT of an $M \times N$ image array f is obtained in the toolbox with function `fft2`, which has the simple syntax:

$$F = \text{fft2}(f)$$

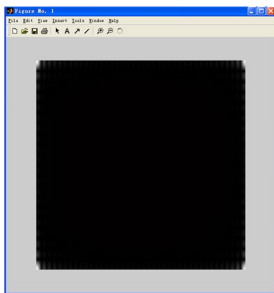
This function returns a Fourier transform that is also of size $M \times N$, with the origin of the data at the top left, and with four quarter periods meeting at the center of the frequency rectangle.

- The Fourier spectrum is obtained by using function `abs`:

$$S = \text{abs}(F)$$

2-D DFT in Matlab(cont.)

- `f=imread( 'Fig0403(a)(image).tif' );`
- `F=fft2(f);`
- `S=abs(F);`
- `imshow(S,[])`

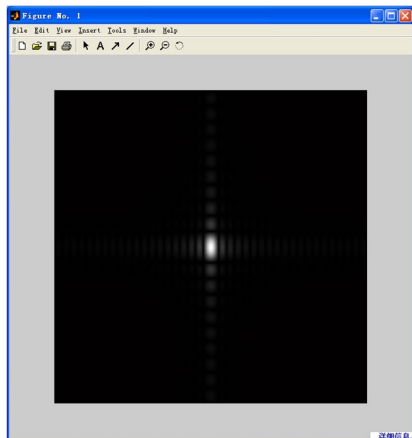


2-D DFT in Matlab(cont.)

- `Fc=fftshift(F);`
- `imshow(abs(Fc),[])`

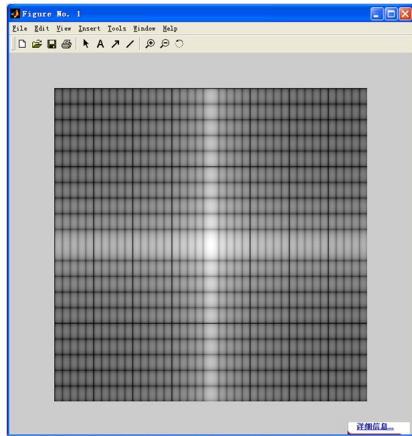
The net result of using `fftshift` is the same as if the input image had been multiplied by $(-1)^{x+y}$ prior to computing the transform.

Note, however, that the two processes are not interchangeable.

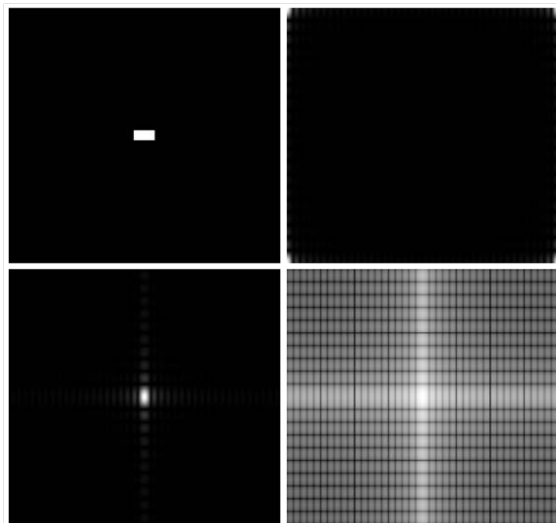


2-D DFT in Matlab(cont.)

- $S2 = \log(1 + \text{abs}(F_c));$
- `imshow(S2,[])`



2-D DFT in Matlab(cont.)



a	b
c	d

FIGURE 4.3

(a) A simple image.
(b) Fourier spectrum.
(c) Centered spectrum.
(d) Spectrum visually enhanced by a log transformation.

2-D DFT in Matlab(cont.)

- We point out that the inverse Fourier transform is computed using function `ifft2`. which has the basic syntax

`f=ifft2(F)`

- In practice, the output of `ifft2` often has **very small imaginary components** resulting from round-off errors. Thus, it is good practice to **extract the real part of the result**.

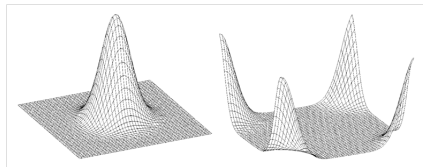
`f=real(ifft2(F))`

Fundamental Concepts

- The foundation for linear filtering in both the spatial and frequency domains is the convolution theorem, which may be written as.

$$f(x, y) * h(x, y) \Leftrightarrow F(u, v)H(u, v)$$

- The idea in frequency domain filtering is to select a filter transfer function that modifies $F(u, v)$ in a specified manner.
- For example, the lowpass filter in figure 4.4



a b

FIGURE 4.4

Transfer functions of (a) a centered lowpass filter, and (b) the format used for DFT filtering. Note that these are frequency domain filters.

Fundamental Concepts(cont.)

- Based on the convolution theorem, we know that to obtain the compute the inverse Fourier transform corresponding filtered image in the spatial domain we simply of the product $H(u, v)F(u, v)$.
- Convolving periodic functions can cause interference of the nonzero periods if the periods are close with respect to the duration of the nonzero parts of the functions. This interference, called *wraparound error*, can be avoided by padding the functions with zeros.
- For example, the lowpass filter in figure 4.4

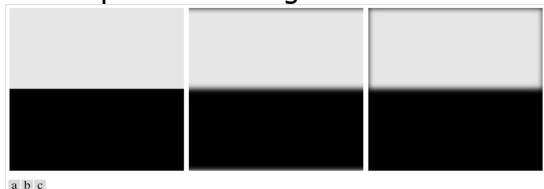


FIGURE 4.5 (a) A simple image of size 256×256 . (b) Image lowpass-filtered in the frequency domain without padding. (c) Image lowpass-filtered in the frequency domain with padding. Compare the light portion of the vertical edges in (b) and (c).

Fundamental Concepts(cont.)

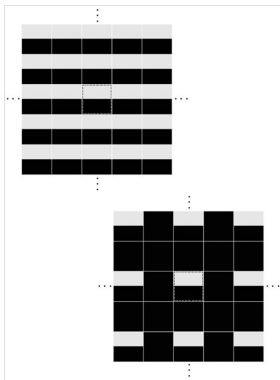


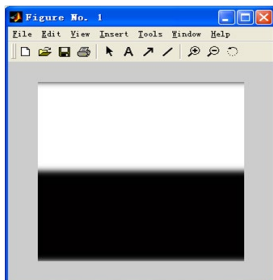
FIGURE 4.6
 (a) Implied, infinite periodic sequence of the image in Fig. 4.5(a). The dashed region represents the data processed by `fft2`. (b) The same periodic sequence after padding with 0s. The thin white lines in both images are shown for convenience in viewing; they are not part of the data.



FIGURE 4.7 Full padded image resulting from `ifft2` after filtering. This image is of size 512 x 512 pixels.

Fundamental Concepts(cont.)

- Image lowpass-filtered in the frequency domain without padding
 - `f=imread('Fig0405(a)(square original).tif');`
 - `[m n]=size(f)`
 - `F=fft2(f);`
 - `H=lpfilter('gaussian',m,n,10);`
 - `G=H.*F;`
 - `g=real(ifft2(G))`
 - `imshow(g,[])`



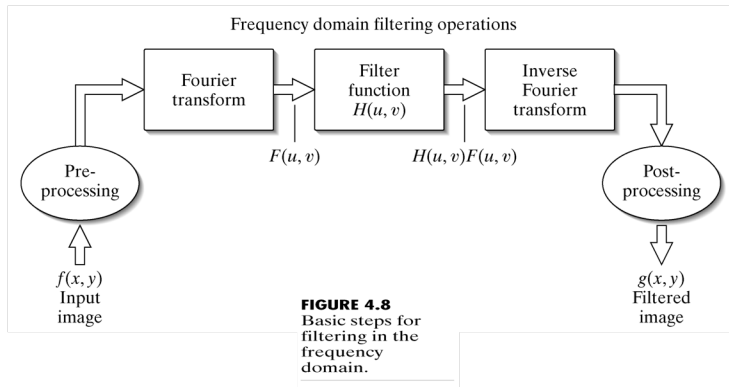
Basic Steps in DFT Filtering

- 1. Obtain the padding parameters using function `paddedsize`:
`PQ=paddedsize(size(f));`
- 2. Obtain the Fourier transform with padding:
`F=fft2(f,PQ(1),PQ(2));`
- 3. Generate a filter function, H , of size $PQ(1) \times PQ(2)$ using any of the methods discussed in the remainder of this chapter.

The filter must be in the format shown in Fig. 4.4(b). If it is centered instead, as in Fig. 4.4(a), let $H = \text{ifftshift}(H)$ before using the filter.

- 4. Multiply the transform by the filter: `G = H.*F;`
- 5. Obtain the real part of the inverse FFT of G : `g=real(ifft2(G));`
- 6. Crop the top, left rectangle to the original size:
`g=g(1:size(f,1),1:size(f,2));`

Basic Steps in DFT Filtering(cont.)



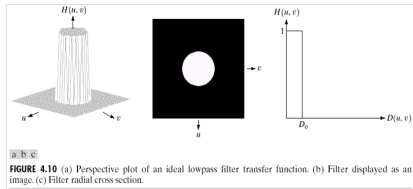
Generating Filters Directly in the Frequency Domain

- Ideal lowpass filter(ILPF)

$$H(u, v) = \begin{cases} 1 & \text{if } D(u, v) \leq D_0 \\ 0 & \text{if } D(u, v) > D_0 \end{cases}$$

where $D(u, v)$: the distance from point (u, v) to the center of the frequency rectangle

$$D(u, v) = [(u - M/2)^2 + (v - N/2)^2]^{\frac{1}{2}}$$

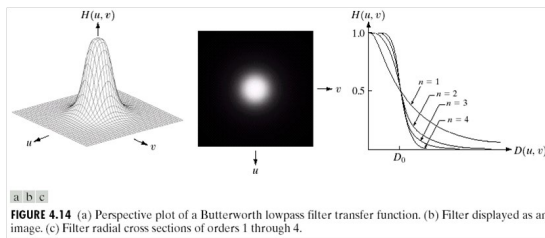


- Create meshgrid arrays for using in implementing Filters in the frequency domain **dftuv**

Generating Filters Directly in the Frequency Domain(cont.)

- Butterworth Lowpass Filters (BLPFs) with order n

$$H(u, v) = \frac{1}{1 + [D(u, v)/D_0]^{2n}}$$



Generating Filters Directly in the Frequency Domain(cont.)

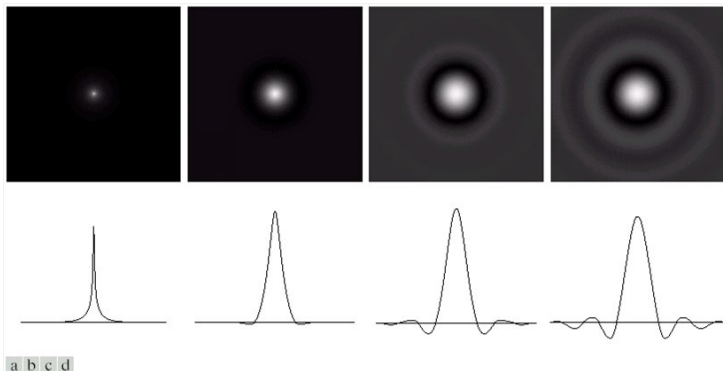
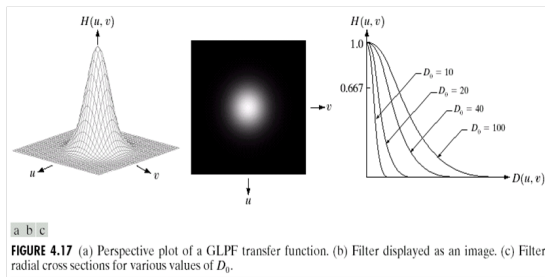
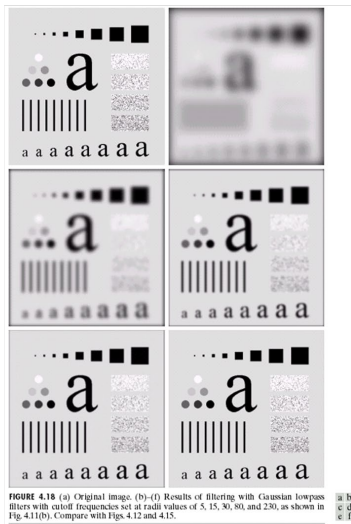
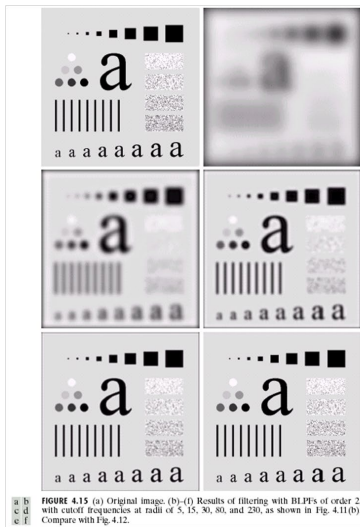


FIGURE 4.16 (a)–(d) Spatial representation of BLPFs of order 1, 2, 5, and 20, and corresponding gray-level profiles through the center of the filters (all filters have a cutoff frequency of 5). Note that ringing increases as a function of filter order.

Gaussian Lowpass Filters (GLPFs)

- $$H(u, v) = e^{-D^2(u, v)/2D_0^2}$$





Examples of Lowpass Filtering

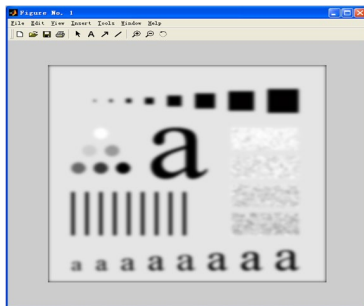


a b c

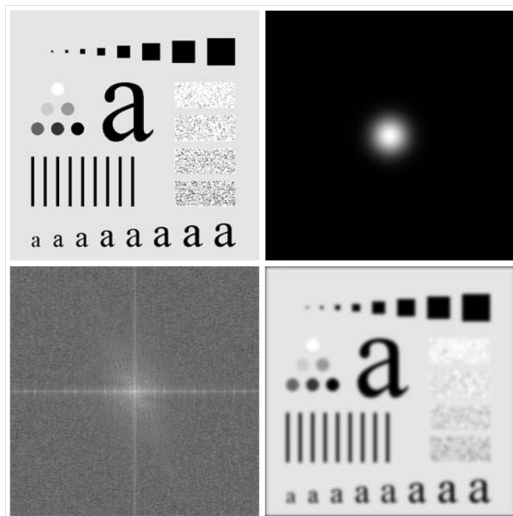
FIGURE 4.20 (a) Original image (1028×732 pixels). (b) Result of filtering with a GLPF with $D_0 = 100$. (c) Result of filtering with a GLPF with $D_0 = 80$. Note reduction in skin fine lines in the magnified sections of (b) and (c).

Generating Filters Directly in the Frequency Domain

- `>>f=imread('Fig0413(a)(original-test-pattern).tif');`
- `>>PQ=paddedsized(size(f));`
- `>>[U V]=dftuv(PQ(1),PQ(2));`
- `>>D0=0.05*PQ(2);`
- `>>F=fft2(f,PQ(1),PQ(2));`
- `>>H=exp(-(U.^2+V.^2)/(2*(D0^2))); H=ifftshift(H);`
- `>>g=dftfilt(f,H);`
- `>>imshow(g,[]);`



Generating Filters Directly in the Frequency Domain(cont.)



a	b
c	d

FIGURE 4.13

Lowpass filtering.

(a) Original image.

(b) Gaussian lowpass filter shown as an image.

(c) Spectrum of (a). (d) Processed image.

Sharpening Frequency Domain Filters

- General high-pass frequency domain filters

$$H_{hp}(u, v) = 1 - H_{lp}(u, v)$$

Why? how to prove it?

Sharpening Frequency Domain Filters

- Ideal highpass filter

$$H(u, v) = \begin{cases} 0 & \text{if } D(u, v) \leq D_0 \\ 1 & \text{if } D(u, v) > D_0 \end{cases}$$

- Butterworth highpass filter

$$H(u, v) = \frac{1}{1 + [D_0/D(u, v)]^{2n}}$$

- Gaussian highpass filter

$$H(u, v) = 1 - e^{-D^2(u, v)/2D_0^2}$$

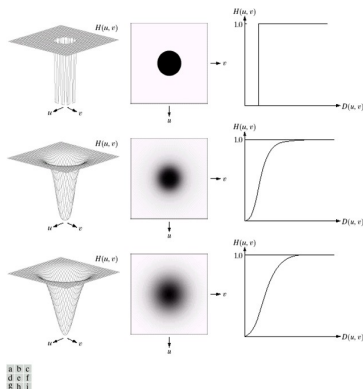


FIGURE 4.22 Top row: Perspective plot, image representation, and cross section of a typical ideal highpass filter. Middle and bottom rows: The same sequence for typical Butterworth and Gaussian highpass filters.

Highpass Filters Spatial Representations

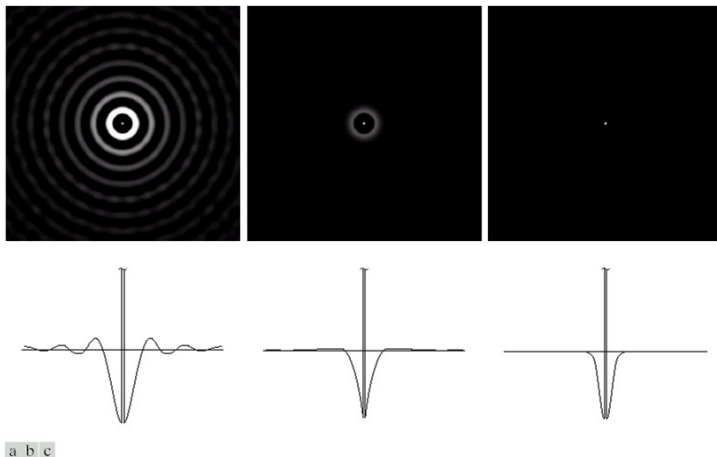
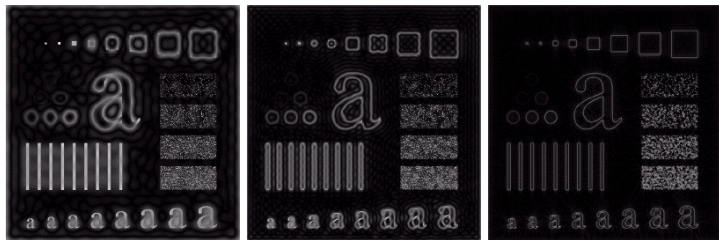


FIGURE 4.23 Spatial representations of typical (a) ideal, (b) Butterworth, and (c) Gaussian frequency domain highpass filters, and corresponding gray-level profiles.

Ideal Highpass Filters

- Ideal highpass filter

$$H(u, v) = \begin{cases} 0 & \text{if } D(u, v) \leq D_0 \\ 1 & \text{if } D(u, v) > D_0 \end{cases}$$



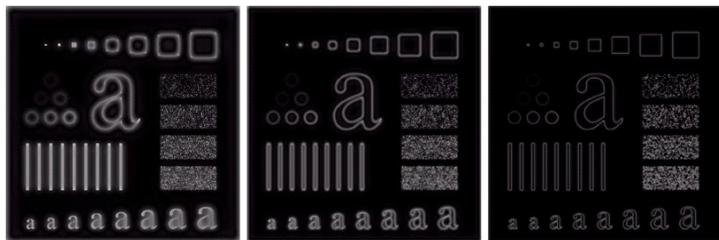
a b c

FIGURE 4.24 Results of ideal highpass filtering the image in Fig. 4.11(a) with $D_0 = 15, 30$, and 80 , respectively. Problems with ringing are quite evident in (a) and (b).

Butterworth Highpass Filters

- Butterworth highpass filter

$$H(u, v) = \frac{1}{1 + [D_0/D(u, v)]^{2n}}$$



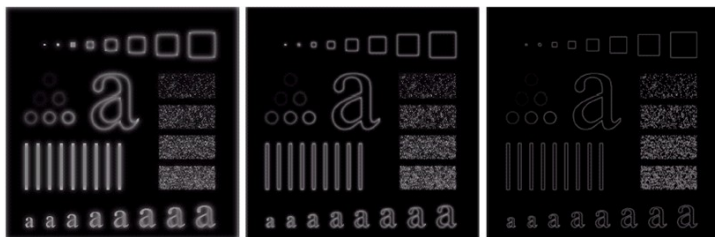
a b c

FIGURE 4.25 Results of highpass filtering the image in Fig. 4.11(a) using a BHPF of order 2 with $D_0 = 15$, 30, and 80, respectively. These results are much smoother than those obtained with an ILPF.

Gaussian Highpass Filters

- Gaussian highpass filter

$$H(u, v) = 1 - e^{-D^2(u, v) / 2D_0^2}$$



a b c

FIGURE 4.26 Results of highpass filtering the image of Fig. 4.11(a) using a GHPF of order 2 with $D_0 = 15$, 30, and 80, respectively. Compare with Figs. 4.24 and 4.25.

Sharpening Frequency Domain Filters

• High-Frequency Emphasis Filtering

$$H_{hfe}(u, v) = a + bH_{hp}(u, v)$$

```
>>f=imread('Fig0419(a)(chestXray_original).tif');
>>PQ=paddedsize(size(f));
>>D0=0.05*PQ(1);
>>HBW=hpfilter('btw',PQ(1),PQ(2),D0,2);
```

```
>>H=0.5+2*HBW;
>>gbf=dftfilt(f,H);
>>ghf=gscale(gbf);
>>ghe=histeq(ghf,256);
>>imshow(ghe);
```

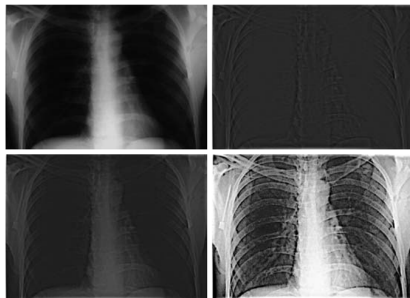


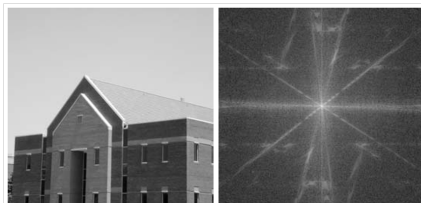
FIGURE 4.19 High-frequency emphasis filtering. (a) Original image. (b) Highpass filtering result. (c) High-frequency emphasis result. (d) Image (c) after histogram equalization. (Original image courtesy of Dr. Thomas R. Gest, Division of Anatomical Sciences, University of Michigan Medical School.)

Obtaining Frequency Domain Filters from Spatial Filters

- Why obtain Frequency Domain Filters from Spatial Filters?
 - Efficiency
 - Meaningful comparisons
- How?
 - How to convert spatial filters into equivalent frequency domain filters;
 - How to compare the results between spatial domain filtering using `imfilter`, and frequency domain filtering using `freqz2`

Obtaining Frequency Domain Filters from Spatial Filters(cont.)

- `>>f=imread('Fig0409(a)(bld).tif');`
- `>>F=fft2(f);`
- `>>S=fftshift(log(1+abs(F)));`
- `>>S=gscale(S);`
- `>>imshow(S)`



a b

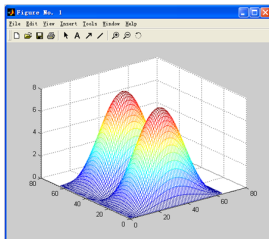
FIGURE 4.9

(a) A gray-scale image. (b) Its Fourier spectrum.

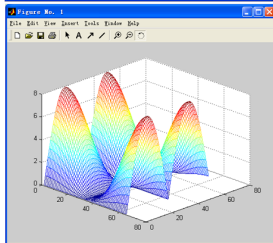
Obtaining Frequency Domain Filters from Spatial Filters

Filters(cont.)

- `h=fspecial( 'sobel' );`
- `H=freqz2(h);`
- `mesh(abs(H))`



- `H1=fftshift(H);`
- `mesh(abs(H1))`
- `view(45,30)`



Obtaining Frequency Domain Filters from Spatial Filters(cont.)

Spatial domain	Frequent domain
<code>gs=imfilter(double(f),h);</code>	<code>PQ=paddedsz(size(f));</code> <code>H=freqz2(h,PQ(1),PQ(2));</code> <code>H1=ifftshift(H);</code> <code>gf=dftfilt(f,H1);</code>



- `d=abs(gs-gf);`
- `max(d(:))`
`ans=`
`5.4015e-012`
- `min(d(:))`
`ans=`
`0`